EX14 - Dictionaries and Sets - Named Configuration

Beginner Step-by-Step Guide

This guide follows the real file `ex14_simulation_config_dict_set.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Replace many configuration/result variables with dictionaries using descriptive keys and validate required keys using sets.

Problem variables / elements

Name	Meaning
config	input dictionary
results	result dictionary
required/provided	sets of keys

Equation or logic on paper

```
missing = required - provided
```

Manual check

If required={"h0","v0","g"} and provided={"h0","g"}, missing={"v0"}.

2. What we already know and what is new

Already seen:

- functions
- lists
- tuples
- validation

New in this exercise:

- dict
- key:value
- config["g"]
- keys()
- set
- set difference
- labelled results

3. New Python concepts

1. Dictionary

Syntax: `config = {"g": 9.81, "dt": 0.1}`

A dict maps keys to values. The key describes what each value means.

2. Key lookup

Syntax: `config["g"]`

Square brackets retrieve the value associated with the key.

3. Set

Syntax: `required = {"h0", "v0", "g", "dt"}`

A set stores unique elements and is useful for membership and set operations.

4. Set difference

Syntax: `missing = required - provided`

Returns items present in required but not in provided.

4. Code step by step

Step 1. Create config

```
config = {
    "h0": 0.0,
    "v0": 50.0,
    "t0": 0.0,
    "g": 9.81,
    "dt": 0.1
}
```

Inputs are grouped and labelled inside one object.

Step 2. Define required keys

```
required = {"h0", "v0", "t0", "g", "dt"}
provided = set(config.keys())
```

config.keys() gets dict keys; set(...) converts them into a set.

Step 3. Find missing keys

```
missing = required - provided
if missing != set():
    return False
```

If any key is missing, missing will not be empty.

Step 4. Use values by key

```
h.append(config["h0"])
euler_step(..., config["g"], config["dt"])
```

We no longer rely on positions: the key name tells us which value is used.

Step 5. Return labelled results

```
results = {
    "max_altitude": max_h,
    "final_velocity": v_f,
    "time_of_flight": t_f,
    "iterations": i
}
```

The caller can use results["max_altitude"] instead of remembering an index.

5. Execution flow

```
config dict
 -> required/provided sets
 -> missing check
 -> simulation using config
 -> results dict
```

6. Expected result / check

Physics remains the same as EX13, but main inputs and results are identified by keys.

7. Common beginner mistakes

- Confusing dict {} with set {}.
- Writing config[g] without g being a key variable.
- Thinking a set is meant to preserve order.
- Subtracting sets in the wrong order.

8. If you remember only five things

- dict maps keys to values.
- config["key"] retrieves a value.
- keys() exposes keys.
- set compares unique groups.
- required - provided finds missing items.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.